

Covert, Anonymous and Video-Bound Provenance: Hiding zk-SNARK Camera Proofs in H.264 CAVLC Trailing-One Signs

Author Name(s) (to be filled in)

Affiliation

City, Country

e-mail

Abstract—Deployed content-provenance systems attach a signed manifest to a media file. The manifest is metadata that platforms strip, it identifies the signing device, and its presence is visible. We hide a zero-knowledge proof of origin inside the H.264 bitstream instead. A camera whose key is a leaf of a public Poseidon Merkle registry produces a 129-byte Groth16 proof that it is registered and vouches for a binding of the video and a message; the proof is embedded, under a separate stego key, into CAVLC trailing-one signs of IDR frames, which keeps every codeword length and the file size. To bind the proof to the stream that carries it, we hash every NAL unit with exactly the keyed carrier bits cleared: the digest is invariant under embedding, while any other edit—another video, truncation, re-encoding, an edited non-carrier bit—changes it. Verifiers need the stego key and the registry root but no camera secret, cannot forge proofs and do not learn which camera signed. We also derive a closed-form distortion model: one sign flip costs $4Q_{\text{step}}^2\kappa$ in pixel energy, $\kappa \in [0.925, 1.057]$, times a propagation gain G , which inverts into a per-frame bit budget for a target PSNR. On 1,200 single flips and 810 frames the direct energy matches the decoder exactly, G does not depend on QP, and its heavy-tailed distribution—a tenth of the carriers causes four fifths of the distortion—predicts frame distortion. A 68-byte message and its proof occupy 26 IDRs with a minimum PSNR-Y of 44.03 dB; proving takes 1.5–1.8 s, verification about 1 s, and an 11-case attack matrix including replay is rejected as expected.

Index Terms—Video steganography, H.264/AVC, CAVLC, zk-SNARK, Groth16, content provenance, anonymous attestation, distortion model.

I. INTRODUCTION

Whether a video was really captured by a trustworthy device has become a practical question for newsrooms, courts and platforms. The industry answer is the C2PA standard [1]: the capture device signs a manifest that binds a hash of the asset to assertions about its origin, and the manifest travels with the file as metadata. This design has three properties that are undesirable in some settings. First, the manifest is *separable*: social networks and many transcoding pipelines strip metadata, which is why C2PA added watermark-based “soft bindings” to recover a lost manifest. Second, it is *identifying*: the signature is verified against a device or vendor certificate, so the verifier learns who signed. Third, it is *visible*: anyone can see that the file carries provenance, and an adversary who wants to suppress it knows exactly which files to target.

Zero-knowledge proofs address the second point. A camera can prove that it belongs to a set of authorized devices without revealing which one, the pattern behind anonymous credentials and Semaphore-style group membership [2], [3]. Recent work applies zk-SNARKs to media provenance, mostly to prove that a published image or video is a permissible transformation of a signed original [4]–[9]. Steganography addresses the first and third points: a message hidden in the coded bitstream survives metadata stripping and container changes, and is invisible to a party that does not hold the key [10]–[12]. Compressed-domain data hiding in H.264 has a long history, including embedding in the sign bits of CAVLC trailing ones, which leaves the bit rate unchanged [13]–[15].

This paper combines the two. The question we answer is: *can a short zero-knowledge proof of anonymous camera membership be hidden inside the H.264 bitstream it vouches for, bound to that exact bitstream, at a quality cost that can be predicted and controlled?* Three technical obstacles stand in the way. (i) A proof embedded into a video changes the video, so a proof that commits to the video’s hash cannot be computed before embedding; the binding has to be defined over a canonical form that embedding leaves unchanged, without opening a gap an attacker can exploit. (ii) The proof must be small enough for a covert channel whose capacity is a few dozen bits per frame, and its verifier must not need the camera’s secret—otherwise any verifier could forge proofs, and the proof would add nothing over a shared-key MAC. (iii) Sign flips in intra-coded frames propagate through intra prediction, so the quality cost is not simply proportional to the number of embedded bits; it must be modeled before an embedding budget can be chosen.

Contributions.

- A *video-bound anonymous camera proof* (Section V): a Circom circuit of 8,737 constraints with three public inputs (registry root and a 256-bit binding split in two halves) proving that Poseidon(s) is a leaf of a depth-16 Poseidon Merkle registry, with Groth16 proofs of 129 bytes.
- A *carrier-masked video digest*: SHA-256 over the RBSP of every NAL unit with exactly the keyed carrier bits cleared. We prove that it is invariant under embedding and that every other edit is detected, either through the

digest or through the extracted proof input (Section VI).

- A *keyed, segment-wise covert channel* on CAVLC trailing-one signs with HKDF-derived schedule and whitening keys, a three-byte frame, and one segment per IDR, which works identically on files and on live streams.
- A *closed-form distortion model* of trailing-one sign embedding, derived from the H.264 dequantization and inverse transform, together with its inverse: the number of flips, and hence the per-IDR bit cap, that keeps a frame above a target PSNR (Section VII).
- An implementation and an evaluation (Sections VIII–IX): 27 encodes at three resolutions, a model validation on sequence-level and single-flip experiments under two encoder configurations, Groth16/PLONK costs, and an 11-case attack matrix.

Scope. The scheme is *fragile* by design: it protects bitstreams that are transferred unchanged (evidence chains, archives, files sent as files). A re-encode destroys both the payload and the digest; robustness to transformations is the subject of the transformation-proof literature [4], [6], [8] and is outside this paper. The proof states that a registered camera vouched for a bitstream, not that the scene is real.

II. RELATED WORK

Data hiding in the H.264 compressed domain. Embedding directly in entropy-coded syntax avoids a full decode–encode cycle. Kapotas *et al.* [13] and Mobasseri and Raikar [14] embed into CAVLC blocks; flipping the sign of a trailing one keeps the codeword length and therefore the bit rate [15]. Because intra prediction reuses reconstructed neighbors, such changes *drift* inside the frame; Ma *et al.* [16] select blocks whose error cannot propagate. We use the same carrier, add a keyed schedule and whitening, and quantify drift with an explicit model instead of excluding blocks.

Fragile watermarking and self-authentication. A classic construction hashes the content without the embedding region, signs the hash and embeds the signature into that region [17], [18]. Our video digest follows this pattern, with two differences that matter for security: the excluded region is *only* the set of keyed carrier positions (not a whole bit plane or block class), and the embedded object is a zero-knowledge proof rather than a signature, so verification reveals neither the signer nor a verification key per device.

Media provenance. C2PA [1] binds signed manifests to assets through hard bindings (hashes) and soft bindings (watermarks or fingerprints). It offers public verifiability and an established certificate infrastructure, but the manifest is separable metadata and the signer is identified. Our proof travels inside the bitstream, is hidden from parties without the stego key, and hides the signer within the registry.

Zero-knowledge proofs for media. PhotoProof [4] proves that an image is derived from a signed original by permissible transformations. ZK-IMG [5] attests images from attested cameras and hides the pre-image of transformations; VerI-TAS [6] and VIMz [7] scale such proofs to high-resolution images, the latter with folding-based SNARKs and signer

anonymity; zk-Cinema [8] extends transformation proofs to video; AIPA [9] adds unlinkable pseudonym certificates. These systems prove *what happened to* a signed asset and keep the proof beside it. We solve a different problem: the proof is hidden *inside* the asset, bound to the exact bitstream, and the signer is anonymous within a registry; we do not support transformations.

Zero knowledge and information hiding. Adelsbach and Sadeghi [19] prove the presence of a watermark in zero knowledge; Toso and Chung [20] combine steganography and an interactive zero-knowledge proof to embed a signature in an image. We embed a non-interactive succinct argument whose statement depends on the cover itself.

Anonymous membership. Proving that a committed key is a leaf of a public Merkle tree [21] without revealing the leaf is the core of Zerocash [2] and Semaphore [3]; Poseidon [22] makes the in-circuit hashing cheap. We use this pattern for a camera registry.

Minimal-distortion steganography. Matrix embedding [23] and syndrome-trellis codes [24], [25] minimize an additive distortion for a given payload. Our model supplies the per-carrier cost such codes need in the CAVLC sign domain.

Table I summarizes the comparison.

III. BACKGROUND

We summarize the parts of H.264/AVC [26]–[28] that the channel and the distortion model rely on, and the zero-knowledge tools of the proof. We restrict attention to the Constrained Baseline profile: progressive 4:2:0, 8-bit, CAVLC entropy coding, flat scaling matrices.

A. Intra coding of an IDR picture

An IDR picture is split into 16×16 macroblocks (MBs) coded in raster order. In an I-slice an MB is either *Intra4×4* (sixteen 4×4 luma blocks, each predicted with one of nine directional modes from reconstructed samples above and to the left), *Intra16×16* (one of four modes for the whole MB: vertical, horizontal, DC, plane), or *I_PCM*. Chroma (8×8 per MB for 4:2:0) uses one of four modes. The encoder codes the prediction residual $e = x - p$; the decoder reconstructs $\hat{x} = \text{clip}(p + \hat{e})$. Because p is computed from *reconstructed* neighbors, a change in $\hat{e}$ of one block alters the prediction of later blocks: this is intra drift.

B. From the DCT to the integer transform

The 4×4 two-dimensional DCT-II is $Y = AXA^\top$ with

$$A = \begin{bmatrix} a & a & a & a \\ b & c & -c & -b \\ a & -a & -a & a \\ c & -b & b & -c \end{bmatrix}, \quad \begin{aligned} a &= \frac{1}{2}, \\ b &= \frac{1}{\sqrt{2}} \cos \frac{\pi}{8}, \\ c &= \frac{1}{\sqrt{2}} \cos \frac{3\pi}{8}. \end{aligned} \quad (1)$$

TABLE I
COMPARISON WITH PROVENANCE AND COMPRESSED-DOMAIN HIDING APPROACHES. ✓: SUPPORTED; -: NOT SUPPORTED; (✓): PARTIALLY.

Approach	In-band (no metadata)	Hidden	Signer anonymous	Bound to exact stream	Survives re-encode	Public verification
C2PA manifest [1]	–	–	–	✓	(✓) ^a	✓
CAVLC fragile watermark [14], [15]	✓	(✓)	–	✓	–	–
Signed digest in content [18]	✓	–	–	✓	–	✓
PhotoProof / ZK-IMG [4], [5]	–	–	(✓)	–	✓ ^b	✓
VIMz / zk-Cinema / AIPA [7]–[9]	–	–	✓	–	✓ ^b	✓
This work	✓	✓	✓	✓	–	– ^c

^a via soft-binding watermarks that recover the manifest. ^b for the transformations the proof system supports. ^c verifiers need the stego key (Section X).

H.264 replaces $c/b \approx 0.414$ by $1/2$ and rescales $b = \sqrt{2/5}$ to keep the rows orthogonal [29]. The transform factors into an integer core and a scaling that is folded into quantization:

$$Y = (C_f X C_f^\top) \odot E_f, \quad C_f = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 2 & 1 & -1 & -2 \\ 1 & -1 & -1 & 1 \\ 1 & -2 & 2 & -1 \end{bmatrix}, \quad (2)$$

where E_f has entries $a^2, ab/2, b^2/4$ according to the parity class of (i, j) : both indices even, both odd, or mixed. The inverse is

$$X = C_i^\top (Y \odot E_i) C_i, \quad C_i = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & \frac{1}{2} & -\frac{1}{2} & -1 \\ 1 & -1 & -1 & 1 \\ \frac{1}{2} & -1 & 1 & -\frac{1}{2} \end{bmatrix}, \quad (3)$$

with E_i entries a^2, ab, b^2 . Only additions and shifts are needed, and encoder and decoder match exactly. We write $b_0, \dots, b_3$ for the rows of C_i ; their squared norms are $\|b_0\|^2 = \|b_2\|^2 = 4$ and $\|b_1\|^2 = \|b_3\|^2 = 5/2$.

C. Quantization and scaling

The quantizer step Q_{step} doubles every six QP values:

$$Q_{\text{step}}(QP) = Q_{\text{step}}(QP \bmod 6) \cdot 2^{\lfloor QP/6 \rfloor}, \quad (4)$$

with $Q_{\text{step}}(0..5) = 0.625, 0.6875, 0.8125, 0.875, 1, 1.125$. The encoder computes $W = C_f X C_f^\top$ and quantizes

$$|Z_{ij}| = (|W_{ij}| MF_{ij} + f) \gg q_{\text{bits}}, \quad q_{\text{bits}} = 15 + \lfloor QP/6 \rfloor, \quad (5)$$

with $MF = 2^{q_{\text{bits}}} PF / Q_{\text{step}}$, $PF \in \{a^2, ab/2, b^2/4\}$, and a dead-zone offset f ($2^{q_{\text{bits}}}/3$ for intra in the reference design; x264 additionally applies trellis and psychovisual decisions). The decoder scales a level c_{ij} by

$$d_{ij} = c_{ij} \cdot v_m(i, j) \cdot 2^{\lfloor QP/6 \rfloor}, \quad m = QP \bmod 6, \quad (6)$$

where $v_m(i, j) \in \{v_{m,0}, v_{m,1}, v_{m,2}\}$ by parity class (even/even, odd/odd, mixed):

$$\begin{array}{c|cccccc} m & 0 & 1 & 2 & 3 & 4 & 5 \\ \hline v_{m,0} & 10 & 11 & 13 & 14 & 16 & 18 \\ v_{m,1} & 16 & 18 & 20 & 23 & 25 & 29 \\ v_{m,2} & 13 & 14 & 16 & 18 & 20 & 23 \end{array} \quad (7)$$

(The standard writes $\text{LevelScale} = 16 v_m$ and a shift by four for flat matrices; the product is the same.) The residual is then $\hat{e} = (C_i^\top D C_i + 32) \gg 6$, where C_i is applied with integer shifts. Note that $Q_{\text{step}}(m) = v_{m,0}/16$: the step size is the even/even scaling factor.

DC paths. In an Intra16×16 MB the sixteen luma DC coefficients form a 4×4 block coded separately (category *LumaDC*), transformed by a 4×4 Hadamard matrix before scaling; the remaining fifteen coefficients of each block are *Luma AC*. For chroma, the four DC coefficients of each 8×8 component form a 2×2 block (*ChromaDC*) with a 2×2 Hadamard transform, and each 4×4 block has fifteen *ChromaAC* coefficients. Chroma uses QP_c , derived from $QP + \text{chroma_qp_index_offset}$ by a table that saturates at 39.

D. CAVLC

After a zig-zag scan, each block of levels is coded as [26], [28]:

- 1) `coeff_token`: the pair (`TotalCoeff`, `TrailingOnes`), from one of four VLC tables chosen by n_C , the rounded mean of `TotalCoeff` of the left and upper blocks;
- 2) one `trailing_ones_sign_flag` per trailing one—the up to three highest-frequency non-zero coefficients equal to ± 1 —in reverse scan order, $0 \mapsto +1, 1 \mapsto -1$;
- 3) the remaining levels (`level_prefix`, `level_suffix`, `adaptive suffixLength`), `total_zeros` and `run_before`.

Proposition 1 (Syntax invariance). *Flipping a `trailing_ones_sign_flag` changes exactly one level from ± 1 to ∓ 1 and leaves the length and the decoded value of every other syntax element of the slice unchanged.*

Proof. The flag is a fixed one-bit field. `TotalCoeff` and `TrailingOnes`, hence the `coeff_token` codeword and the n_C context of later blocks, do not depend on signs. The initial `suffixLength` depends only on `TotalCoeff` and `TrailingOnes`, its updates only on the non-trailing-one levels, and the magnitude adjustment of the first non-trailing-one level only on `TrailingOnes`. `total_zeros` and `run_before` depend

on positions only. Hence every later bit is parsed at the same offset with the same meaning. $\square$

Consequently the edited stream is a valid stream of the same RBSP length, and the set of trailing-one sign positions (the *candidates*) is the same in the cover and the stego stream. We use the *first* sign flag of each block that has at least one trailing one, i.e. the highest-frequency trailing one.

E. NAL units and emulation prevention

An Annex-B stream is a sequence of NAL units separated by start codes 00 00 01. Each NAL unit has a one-byte header (forbidden_zero_bit, nal_ref_idc, nal_unit_type) followed by the EBSP: the RBSP with an emulation-prevention byte 03 inserted after every 00 00 that precedes a byte ≤ 03 . Bit offsets in this paper are RBSP offsets. A sign flip can add or remove one emulation-prevention byte, which changes the file size by one byte but no RBSP offset.

F. zk-SNARKs, Groth16 and Poseidon

A statement “ $\exists w : \mathcal{R}(x, w)$ ” is compiled to a rank-1 constraint system $(Az) \circ (Bz) = Cz$ over the BN254 scalar field [30], $z = (1, x, w)$. Groth16 [31] proves satisfiability with $\pi = (A, B, C) \in \mathbb{G}_1 \times \mathbb{G}_2 \times \mathbb{G}_1$, verified with one pairing-product equation; it is complete, knowledge-sound and perfectly zero-knowledge in the generic group model. The common reference string comes from a universal Phase 1 (Powers of Tau) and a circuit-specific Phase 2; the multi-party ceremony is secure if one participant is honest [32]. A proof compresses to 129 bytes: the x -coordinates of A , B (two $\mathbb{F}_q$ elements) and C , plus one byte of sign flags. PLONK [33] trades a larger proof for a universal setup. Poseidon [22] is a sponge hash over the scalar field designed for few constraints; we use it for public keys and Merkle nodes. Circuits are written in Circom [34] and proven with snarkjs [35].

IV. SYSTEM AND THREAT MODEL

Parties. A *registry authority* publishes the root ρ of a Merkle tree of camera public keys. A *camera* holds a secret s with Poseidon(s) in the tree. A *sender* (the camera or an embedder it trusts) holds a 32-byte stego key K shared with the intended verifiers, and turns a cover H.264 stream into a stego stream carrying a message m and a proof. The *channel* delivers the bitstream, possibly through a party that inspects or edits it. A *verifier* knows K , ρ and the Groth16 verification key, and accepts or rejects.

Two secrets, two roles. K hides and locates the payload; s authorizes it. Holding K lets a verifier extract and check a proof but not create one, which is the point of using a zero-knowledge proof instead of a MAC under K .

Adversary. The adversary sees stego streams and may know K (a curious or malicious verifier) or not (a channel observer). It may edit, truncate, splice or re-encode streams, move a valid payload into another stream, change the message, register its own cameras in a tree of its choice, and obtain proofs from honest cameras for streams of its choice. It does not know the

secret of any honest registered camera and cannot break SHA-256, Poseidon, HMAC-SHA256 or the Groth16 assumptions.

Goals.

- G1 *Validity*: an honest stego stream is a valid H.264 stream of the same syntax as the cover.
- G2 *Binding*: if the verifier accepts a stream V' with message m' , then a registered camera produced a proof for exactly (V', m') up to the carrier bits that hold the payload.
- G3 *Unforgeability*: without the secret of a registered camera, no adversary—including one holding K —makes the verifier accept a (stream, message) pair that no registered camera vouched for.
- G4 *Anonymity*: the verifier learns nothing about which registered camera produced the proof.
- G5 *Covertiness (empirical)*: a party without K cannot locate the payload, and first-order sign statistics do not reveal embedding.

Non-goals. Robustness to transcoding or editing; proving that the camera recorded a real scene (a camera filming a screen produces a valid proof); verification without K ; resistance to modern learned steganalysis, which we have not evaluated; protection of the camera secret itself, which in our prototype lives in software.

V. DESIGN

Fig. 1 shows the data flow. The sender computes the video digest of the cover under K , binds it with the message, obtains a proof from the camera, packs message and proof into a payload, frames it and embeds the frame into the trailing-one signs selected by a keyed schedule. The verifier reverses the channel, recomputes the digest from the stream it received and verifies the proof against the registry root.

A. Keys

From the 32-byte stego key K the channel derives two subkeys with HKDF-SHA256 [36]:

$$PRK = \text{HMAC}(\text{"zkstego-cavlc-v2-salt"}, K), \quad (8)$$

$$k_{\text{sch}} = \text{Expand}(PRK, \text{".../schedule"}, 32), \quad (9)$$

$$k_{\text{wh}} = \text{Expand}(PRK, \text{".../whitening"}, 32). \quad (10)$$

The camera secret s is an independent uniformly random non-zero element of the BN254 scalar field; its public key is $pk = \text{Poseidon}(s)$.

B. Candidates and segments

Each IDR picture together with its preceding SPS and PPS forms a *segment*. A native parser decodes the CAVLC syntax of the IDR slice and lists one candidate per 4×4 (or 2×2 chroma DC) block that has a trailing one: the RBSP offset of its first trailing_ones_sign_flag. A candidate is identified by the string $id = \text{nal:mb:category:block:bit}$ of its analysis NAL index, MB address, category (LumaDC, Luma4x4/AC, ChromaDC, ChromaAC), block index and

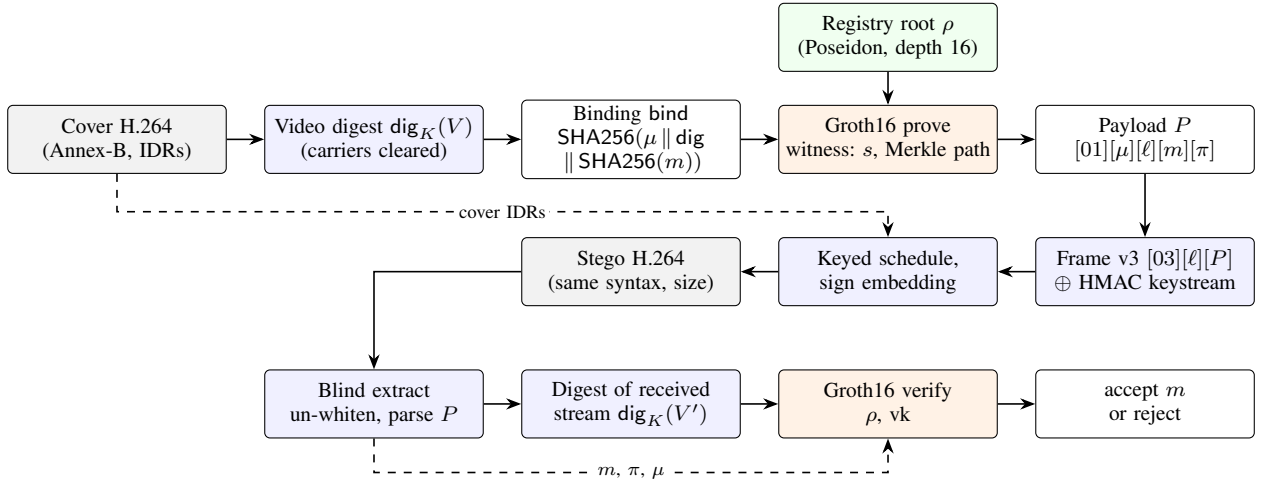


Fig. 1. Data flow. Blue: covert channel under the stego key K ; orange: zero-knowledge proof; the verifier never sees s .

RBSP bit offset. By Proposition 1 the identifiers are the same before and after embedding.

C. Keyed schedule

Within segment j with candidate set $\mathcal{C}_j$, every candidate gets the score $\sigma(id) = \text{HMAC}(k_{\text{sch}}, id)$; candidates are sorted by (σ, id) and the first $n_j = \min(\text{cap}, |\mathcal{C}_j|)$ are selected. Frame bits are assigned to segments in order: segment j carries frame bits $[\sum_{i < j} n_i, \sum_{i \leq j} n_i]$ until the frame ends. The schedule depends only on K , the stream syntax and the per-IDR cap (64 bits by default), so a receiver reconstructs it from the stego stream alone, and the same procedure runs on a live stream as each IDR arrives.

D. Frame and whitening

The channel frame (protocol version 3) is

$$F = 0 \times 03 \parallel \text{u16}(|P|) \parallel P, \quad (11)$$

with no MAC. Bit i of F (MSB first, counted across segments) is XORed with bit i of the keystream $\text{HMAC}(k_{\text{wh}}, \text{u64}(0)) \parallel \text{HMAC}(k_{\text{wh}}, \text{u64}(1)) \parallel \dots$ and written as the sign of the i -th scheduled candidate. Whitening makes the embedded bits pseudo-random, so about half of the scheduled signs actually change, and a wrong key yields a random version byte and length, which the extractor rejects. Authentication of the payload is left entirely to the proof; an earlier version carried a 16-byte HMAC tag, which cost 128 bits (two IDRs at the default cap) and added nothing once the proof binds the message.

E. Camera registry

The registry is a binary Merkle tree of depth 16 whose leaves are camera public keys (unused leaves are 0) and whose internal nodes are Poseidon(ℓ, r). Verifiers trust only the root ρ ; a camera's membership witness is its sibling path $(sib_t, \beta_t)_{t < 16}$.

F. Carrier-masked video digest

Let $\Gamma_K(V)$ be the set of (NAL, RBSP bit) positions that the schedule of K selects to carry the $L_F = 8|F|$ frame bits of a payload of the given length. With the domain string $\text{dom}_v = \text{"zkstego/video-digest/v1"}$, the digest is

$$\text{dig}_K(V) = \text{SHA256}(\text{dom}_v \parallel \text{u32}(L_F) \parallel \text{u32}(\text{cap}) \parallel \parallel_{u \in V} \text{u32}(1 + |r'_u|) \parallel h_u \parallel r'_u), \quad (12)$$

where for each NAL unit u in file order h_u is its header byte and r'_u its RBSP with every bit in $\Gamma_K(V)$ cleared. The schedule depends on the syntax only, so $\Gamma_K(V) = \Gamma_K(V^*)$ for a cover V and its stego V^* .

G. Binding, statement and payload

For a mode $\mu \in \{0, 1\}$ (0: video-bound; 1: message-only, Section X) the binding, with $\text{dom}_b = \text{"zkstego/proof-binding/v1"}$, is

$$\text{bind} = \text{SHA256}(\text{dom}_b \parallel \mu \parallel \text{dig}_K(V) \parallel \text{SHA256}(m)), \quad (13)$$

split into two 128-bit field elements $(\text{bind}_{\text{hi}}, \text{bind}_{\text{lo}})$. The circuit proves

$$\mathcal{R} = \{(\rho, \text{bind}_{\text{hi}}, \text{bind}_{\text{lo}}; s, sib, \beta) : \text{MerkleRoot}(\text{Poseidon}(s), sib, \beta) = \rho \wedge \beta_t \in \{0, 1\}\}. \quad (14)$$

The binding halves are public inputs that enter the R1CS through a squaring constraint, so a proof verifies only for the binding it was produced for. The circuit has 8,737 constraints and three public inputs. The payload is

$$P = 0 \times 01 \parallel \mu \parallel \text{u16}(|m|) \parallel m \parallel \pi, \quad |\pi| = 129, \quad (15)$$

so $L_F = 8(|m| + 136)$; a 68-byte message gives $|P| = 201$, $|F| = 204$ bytes, 1,632 bits, i.e. 26 IDRs at 64 bits each.

Algorithm 1 Embed (sender)

Require: cover V , message m , stego key K , camera $(s, path)$, cap

- 1: $L_F \leftarrow 8(|m| + 136)$; $\Gamma \leftarrow \text{schedule}(V, K, L_F, cap)$
- 2: $\delta \leftarrow \text{dig}_K(V)$ $\triangleright$ Eq. (12)
- 3: $\text{bind} \leftarrow \text{SHA256}(\text{dom} \parallel 0 \parallel \delta \parallel \text{SHA256}(m))$
- 4: $\pi \leftarrow \text{Groth16.Prove}(\rho, \text{bind}; s, path)$
- 5: $F \leftarrow 03 \parallel \text{u16}(|P|) \parallel P$ with $P = 01 \parallel 0 \parallel \text{u16}(|m|) \parallel m \parallel \pi$
- 6: **for** $i = 0$ **to** $L_F - 1$ **do**
- 7: set the sign at $\Gamma[i]$ to $F_i \oplus ks_i$
- 8: **end for**
- 9: **return** V^*

Algorithm 2 Verify (receiver)

Require: stream V' , K , root ρ , verification key vk , cap

- 1: read the first 24 scheduled bits, un-whiten; **reject** unless version = 3 and length is admissible
- 2: read and un-whiten the remaining bits; parse P ; **reject** if malformed
- 3: **if** $\mu = 1$ and message-only proofs are not allowed **then reject**
- 4: **end if**
- 5: $\delta' \leftarrow \text{dig}_K(V')$ with L_F from the header
- 6: $\text{bind}' \leftarrow \text{SHA256}(\text{dom} \parallel \mu \parallel \delta' \parallel \text{SHA256}(m))$
- 7: **return** $\text{Groth16.Verify}(vk, (\rho, \text{bind}'_{hi}, \text{bind}'_{lo}), \pi)$

H. Algorithms

Algorithms 1 and 2 summarize sender and receiver. The service exposes the verifier as an asynchronous job whose outcomes are `payload_not_found`, `malformed_proof_payload`, `video_binding_required`, `video_digest_unavailable`, `proof_invalid` or acceptance with `video_bound`; an extraction-only job returns the payload marked `verified: false`.

VI. SECURITY ANALYSIS

We argue goals G1–G5 informally under standard assumptions: SHA-256 and the Poseidon Merkle tree are collision resistant, Poseidon is one-way on uniformly random inputs, HMAC-SHA256 is a PRF, and Groth16 is perfectly zero-knowledge and weakly simulation-extractable: an adversary that has seen proofs for some public inputs can re-randomize them, but cannot produce an accepting proof for a *new* public input without knowing a witness [37].

Proposition 2 (Validity, G1). *Embedding changes only bits in $\Gamma_K(V)$; the stego stream parses into the same syntax elements with the same lengths as the cover, and only the selected trailing-one levels change sign.*

Proof. Every bit in $\Gamma_K(V)$ is a `trailing_ones_sign_flag`; apply Proposition 1 once per flip, in any order. $\square$

Lemma 1 (Digest invariance). *For a cover V and its stego stream V^* under key K , $\text{dig}_K(V^*) = \text{dig}_K(V)$.*

Proof. By Proposition 2 both streams have the same NAL units, headers and RBSP lengths (emulation-prevention bytes are not part of the RBSP), and the same candidates, so the schedule yields $\Gamma_K(V^*) = \Gamma_K(V)$. Their RBSPs differ only inside this set, which Eq. (12) clears in both. $\square$

Lemma 2 (Syntax determines the mask). *If $\text{dig}_K(V') = \text{dig}_K(V)$ for a stream V' , then, except with negligible probability, V' and V have the same NAL units and RBSP lengths and differ only at positions in $\Gamma_K(V) = \Gamma_K(V')$.*

Proof. Collision resistance of SHA-256 gives equal canonical strings, hence equal headers and lengths, and RBSPs that agree outside $\Gamma_K(V) \cup \Gamma_K(V')$. Consider the first differing bit in parsing order. Both streams parse identically up to it, so it has the same syntactic role in both; it lies in one of the two masks, hence it is a trailing-one sign flag in both. By Proposition 1 the parse of the rest is unaffected, and induction over the differing bits shows that V and V' have identical syntax and candidates. The schedule depends only on syntax and K , so the masks are equal. $\square$

Theorem 1 (Binding, G2). *Suppose the verifier accepts (V', m') in mode 0 and the adversary does not know the secret of any registered camera. Then, except with negligible probability, some registered camera produced a proof for a pair (V, m) with $m' = m$ and V' equal to V (or to its honest stego stream) outside the carrier set $\Gamma_K(V)$.*

Proof. Acceptance means a valid proof for (ρ, bind') with $\text{bind}' = \text{SHA256}(\text{dom} \parallel 0 \parallel \text{dig}_K(V') \parallel \text{SHA256}(m'))$. If no honest proof was ever produced for bind' , weak simulation-extractability yields a witness $(s', path)$ whose leaf $\text{Poseidon}(s')$ opens to ρ ; by collision resistance of the tree the leaf is a registered key, and s' is then a preimage of an honest camera's public key, contradicting one-wayness. Hence an honest camera proved $\text{bind}' = \text{bind}$ for some (V, m) . Collision resistance of SHA-256 gives $\text{SHA256}(m') = \text{SHA256}(m)$, so $m' = m$, and $\text{dig}_K(V') = \text{dig}_K(V)$; Lemma 2 concludes. $\square$

Remark 1 (What the carriers can still hold). Theorem 1 leaves the carrier bits free, but those bits *are* the payload: they must parse as a frame whose message is m and whose proof verifies for bind . An adversary can therefore at most replace π by a re-randomized proof for the same statement. Flipping a carrier changes the extracted payload and is rejected (Section IX); flipping any other bit changes the digest.

Corollary 1 (Attacks covered). *Each of the following yields rejection: embedding a valid payload into another video (the digest of the other video differs); dropping, appending or truncating NAL units; editing any non-carrier bit, including a trailing-one sign in a carrier IDR or any bit of a later frame; re-encoding; replacing the message; a proof made by a camera outside the trusted registry (it opens to a different root).*

Proposition 3 (Unforgeability with the stego key, G3). *Holding K does not help produce an accepting proof for a new binding: K enters only the schedule, the whitening and the mask, all of which are public to the verifier anyway, while acceptance requires a proof for bind, which by the argument of Theorem 1 requires a registered secret.*

This is the property a MAC-based design lacks: with a tag under K , every verifier can authenticate arbitrary content.

Proposition 4 (Anonymity, G4). *The verifier's view— ρ , bind and π —has the same distribution for every registered camera.*

Proof. ρ and bind do not depend on which leaf is used. By perfect zero knowledge, π can be simulated from the statement alone, so its distribution does not depend on the witness. $\square$

Anonymity is relative to the registry: a registry of one camera identifies it.

Coverttness (G5). Without K the schedule is a pseudo-random subset of the candidates and the embedded bits are pseudo-random; with a wrong key the version byte matches with probability 2^{-8} and the length must additionally be admissible, after which the payload parser and the proof reject. Sign flips do not change the bit rate, the codeword lengths or the file size apart from rare emulation-prevention bytes. We only measure first-order statistics: the share of negative trailing ones does not change significantly (Section IX). Calibrated or learned steganalysis is future work, and we do not claim security against it.

Mode 1. A live stream embeds its proof in the first IDRs, before the rest exists, so the proof can only bind the message. Theorem 1 then holds for m but not for the video, and a mode-1 payload can be replayed into another stream. Verifiers reject mode 1 unless explicitly configured otherwise.

VII. A DISTORTION MODEL FOR TRAILING-ONE SIGN EMBEDDING

The forward direction of the channel—choose candidates, embed, measure PSNR—does not tell a designer how many bits a frame can take before its quality drops below a target. This section derives the distortion of one flip in closed form, models its propagation, and inverts the result into a per-IDR bit budget. All energies are sums of squared errors (SSE) of 8-bit samples; $\text{PSNR} = 10 \log_{10}(255^2 N_{\text{px}}/\text{SSE})$ for a plane of N_{px} samples.

A. Direct distortion of one flip

Lemma 3 (Single flip). *Flipping the sign of a trailing one at raster position (i, j) of a 4×4 block coded with quantization parameter QP changes the reconstructed residual of that block, before rounding and clipping, by an error of energy*

$$E_{ij} = 4 Q_{\text{step}} (QP)^2 \kappa_m(i, j), \quad (16)$$

$$\kappa_m(i, j) = \frac{v_m(i, j)^2 \|b_i\|^2 \|b_j\|^2}{16 v_{m,0}^2}, \quad (17)$$

with $m = QP \bmod 6$. $\kappa_m(i, j) = 1$ for the even/even class (including the DC position) and $\kappa_m(i, j) \in [0.925, 1.057]$ for all positions and all m .

Proof. The level changes by $\Delta c = \mp 2$. By Eq. (6) the scaled coefficient changes by $\Delta d = \mp 2 v_m(i, j) 2^{\lfloor QP/6 \rfloor}$ and no other coefficient changes. By linearity of Eq. (3) before the final rounding, the residual error is $\Delta e(x, y) = \Delta d b_i(x) b_j(y)/64$, whose energy is $(\Delta d/64)^2 \|b_i\|^2 \|b_j\|^2$. Substituting $Q_{\text{step}} = v_{m,0} 2^{\lfloor QP/6 \rfloor}/16$ gives Eq. (16). The bounds follow by evaluating the six tables: $\kappa \in \{1, 1.056\}$, $\{1, 1.012, 1.046\}$, $\{0.925, 0.947, 1\}$, $\{1, 1.033, 1.054\}$, $\{0.954, 0.977, 1\}$ and $\{1, 1.014, 1.020\}$ for $m = 0, \dots, 5$. $\square$

The near-constancy of κ is not a coincidence: the scaling factors v_m were designed to compensate the unequal norms of the integer basis [29], so the scaled transform is close to orthonormal and, by Parseval, a level change of ± 2 costs $(2Q_{\text{step}})^2$ wherever it occurs. The energy depends on the quantizer only; it does not depend on the image, the prediction mode or the frequency.

Lemma 4 (DC paths). *A flip in an Intra16×16 LumaDC block or a ChromaDC block has, before rounding, the same total energy $4Q_{\text{step}}^2$ (with QP_c for chroma), spread uniformly over the sixteen (resp. four) 4×4 blocks of the MB component.*

Proof. LumaDC: the Hadamard output changes by ± 2 in all sixteen positions, and the DC scaling $(f \cdot 16 v_{m,0} 2^{\lfloor QP/6 \rfloor})/64$ turns this into a change of $v_{m,0} 2^{\lfloor QP/6 \rfloor}/2$ in the DC coefficient of each block, i.e. a constant residual change of $v_{m,0} 2^{\lfloor QP/6 \rfloor}/128$ over each block; the total energy is $256 (v_{m,0} 2^{\lfloor QP/6 \rfloor}/128)^2 = 4Q_{\text{step}}^2$. ChromaDC: the 2×2 Hadamard output changes by ± 2 in four positions, the scaling $(f \cdot 16 v_{m,0} 2^{\lfloor QP_c/6 \rfloor})/32$ gives $v_{m,0} 2^{\lfloor QP_c/6 \rfloor}$ per block, and four blocks of sixteen samples give the same total. $\square$

Additivity. The schedule uses one candidate per block, so two flips in the same block can only be a DC-path flip and an AC flip of the same MB component. Their errors are orthogonal (an AC basis function has zero mean, a DC change is constant over the block), so the direct energies add exactly. Rounding by $(\cdot + 32) \gg 6$ adds a bounded perturbation; in the demo block of Appendix A the exact rounded energy is 128 against a model value of 121.

B. Propagation

Ignoring clipping and the rounding of prediction, intra decoding is affine in the residual: in decoding order every block is its prediction (a linear function of earlier reconstructed samples) plus its residual. A direct error δ_k of flip k therefore becomes a total error $\mu_k = (I - \mathcal{P})^{-1} \delta_k$, where $\mathcal{P}$ is the strictly lower-triangular prediction operator of the frame, and deblocking adds a further, data-dependent smoothing. We define the *propagation gain*

$$G_k = \frac{\|\mu_k\|^2}{4Q_{\text{step}} (QP_k)^2}, \quad (18)$$

which captures drift through intra prediction and deblocking ($G_k \approx \kappa_k$ when nothing propagates; rounding and clipping can push it slightly below). For a set of flips the total error is $\sum_k \mu_k$; cross terms $\langle \mu_k, \mu_l \rangle$ carry the signs of the flipped levels, which are those of the cover coefficients, and are zero-mean over the random keyed choice of carriers. Hence

$$\mathbb{E}[\text{SSE}] \approx \sum_k 4 Q_{\text{step}} (QP_k)^2 G_k. \quad (19)$$

G_k depends on the prediction structure—MB type, prediction modes, block position, whether deblocking is on—which is why it is measured rather than derived (Section IX).

C. Forward model and inverse

With constant QP, N flips in the luma plane and a frame-level gain $\bar{G}$ (the geometric mean of G_k fitted on data),

$$\text{PSNR}_Y \approx 10 \log_{10} \frac{255^2 WH}{4 \bar{G} N Q_{\text{step}} (QP)^2}. \quad (20)$$

Three rules of thumb follow: doubling the number of flips costs 3.01 dB, raising QP by six costs 6.02 dB, and four times more pixels gain 6.02 dB at the same number of flips. Inverting Eq. (20) for a target T :

$$N_{\text{max}}(T) = \frac{255^2 WH 10^{-T/10}}{4 \bar{G} Q_{\text{step}} (QP)^2}, \quad \text{cap}(T) = \frac{N_{\text{max}}(T)}{p f_Y}, \quad (21)$$

where $p = 1/2$ is the probability that a whitened bit changes the sign and f_Y the share of luma candidates (a uniformly random schedule selects luma carriers at that rate). Chroma planes have their own budget with QP_c and the chroma gain.

Receiver computability. A per-IDR cap is usable only if the receiver derives the same value from the stego stream. Eq. (21) uses W , H , the slice QP, f_Y and a protocol constant $\bar{G}$; by Proposition 1 all of them are identical in the cover and the stego stream. An adaptive cap therefore needs no side information. The same constraint applies to any cost-aware schedule: costs may depend on positions, categories, MB types, prediction modes and coefficient magnitudes, but never on the sign values that carry data.

D. Towards minimal-distortion embedding

Eq. (16) supplies an additive per-carrier cost $\rho_k = 4Q_{\text{step}}(QP_k)^2 \kappa_k \hat{G}_k$, with $\hat{G}_k$ predicted from invariant features (e.g. category and MB type). For a payload of L bits, the distortion-limited sender of [24], [25] changes carrier k with probability $p_k = 1/(1 + e^{\lambda \rho_k})$, where λ solves $\sum_k h(p_k) = L$ (h the binary entropy) and the expected distortion is $\sum_k p_k \rho_k$; syndrome-trellis codes come close to this bound. Even without costs, matrix embedding reduces flips per bit: the present channel spends $1/2$ expected flip per bit, while a Hamming code with $r = 3$ embeds three bits into seven carriers with $1 - 2^{-3} = 0.875$ expected flips, i.e. 0.29 flips per bit, a $10 \log_{10}(0.5/0.29) \approx 2.3$ dB gain at equal payload. Candidates are plentiful (thousands per IDR against 64 used), so the larger carrier count is affordable. We leave both to future work and use the model here to explain and set the cap.

VIII. IMPLEMENTATION

Native codec layer (C++17). A self-contained H.264 Baseline parser splits Annex-B streams, removes emulation prevention, parses SPS, PPS and I-slices down to every CAVLC element, and lists candidates. The same library implements the segment protocol for files and for byte streams, the carrier-masked digest, and re-emission of patched NAL units with emulation prevention. Command-line tools read the 32-byte key from standard input, never from the command line: `embed-stream-auth-stdin`, `extract-stream-auth`, `video-digest`, live variants on `stdin/stdout`, and an inspection tool that prints NAL tables, per-block coefficients and per-segment candidates as JSON.

Proof layer. The circuit is written in Circom 2.1 with `circumlib`'s Poseidon; proofs are generated and verified with `snarkjs` 0.7.6 on Node.js 22. The setup script uses the public Hermez Powers of Tau of size 2^{16} [38], pinned by its published BLAKE2b hash, followed by two Phase-2 contributions with fresh entropy, a final beacon and a `zkey verify`. Both contributions run on one machine, so this is a demonstration ceremony; a deployment needs independent contributors. The verifier and the registry code (Python) reproduce the native digest byte for byte; a Python reference of the schedule and whitening is checked against the native codec in the test suite.

Service. A FastAPI application exposes asynchronous HTTP jobs (`embed`, `extract`, `verify`) and a WebSocket endpoint that pipes an Annex-B byte stream through the live embedder. Keys and messages reach native processes only through standard input; job records store neither; artifacts are downloadable once. The `verify` job loads the registry root from configuration and accepts message-only proofs only when explicitly enabled.

Testing. The test runner groups 151 checks into twelve phases (proof, native contract, CLI fixtures, VLC tables, service, HTTP/WebSocket, manifest security, scheduler, benchmark analysis including the distortion model, trust interfaces, demo arithmetic, demo smoke run), plus a CTest suite for the native library. A terminal walkthrough performs 60 automated cross-checks, e.g. that FFmpeg's decoder and our parser agree on every MB type and QP, that a Python re-implementation of dequantization, inverse transform and intra prediction reproduces FFmpeg's samples, and that the Python and native digests agree.

IX. EVALUATION

A. Setup

All measurements ran on one laptop (Intel Core i7-12700H, 14 cores/20 threads, 25.4 GB RAM, Windows 11) with FFmpeg 8.0.1 (x264 core 165), Node.js 22.20 and `snarkjs` 0.7.6. Sources are nine Xiph CIF sequences [39] (`akiyo`, `city`, `coastguard`, `container`, `deadline`, `football`, `two foreman` variants, `hall_monitor`), 30 frames each, encoded at their native 352×288 and Lanczos-upscaled to 640×480 and 1280×960 (upscaled sequences test scaling, not native high-resolution content). The benchmark encoder is x264 *ultrafast*/zerolatency, Constrained Baseline, QP 22 for P frames (the I-slice QP

TABLE II

PER-FRAME QUALITY OVER THE 26 CARRYING FRAMES OF 9 SEQUENCES
PER RESOLUTION.

	CIF	640 × 480	1280 × 960
Cover→stego PSNR-Y, min [dB]	46.71	44.03	44.72
Cover→stego PSNR-Y, mean [dB]	57.44	59.69	62.64
Cover→stego SSIM, min	0.99871	0.99895	0.99960
Source→stego PSNR-Y, min [dB]	41.48	41.62	42.94
Source→stego PSNR-Y, mean [dB]	45.57	46.43	48.11
Changed signs per case (mean)	808	823	816
Candidates per case (mean)	149,374	420,193	1,287,323

is 19), all-intra, one slice per IDR; this preset uses only Intra16×16 MBs and no deblocking. Each case embeds a 68-byte message with its proof (payload 201 B, 1,632 frame bits) at the default cap of 64 bits per IDR, i.e. into 26 of the 30 IDRs, under a fresh stego key and an 8-camera registry. Wall times include process start-up (Node.js for snarkjs).

B. Functional results

All 27 cases passed every acceptance check: native capacity scan, embedding, strict FFMpeg decoding (`-xerror`), blind extraction of the exact payload, a video-bound proof verified against the registry root with the digest recomputed from the stego file, rejection under a wrong key, and zero sign changes outside the keyed schedule (checked position by position). The stego file has exactly the size of the cover in all 27 cases. A separate end-to-end run on 300 CIF frames passed five of five trials.

C. Embedding quality

Table II reports PSNR-Y and SSIM [40] per frame, comparing the clean decode of the cover with the decode of the stego stream (incremental embedding distortion), and from the original source (encoding plus embedding). About half of the 1,632 scheduled signs change (808–823 on average), i.e. about 31 per carrying frame. No frame falls below the common 40 dB reference. The encoder’s own loss dominates: on average, coverostego PSNR is 12–15 dB higher than sourceostego PSNR.

D. Validating the distortion model

We test the model of Section VII in three experiments (scripts and raw data in the artifact): (B) single flips decoded in isolation, which measure the direct energy and the gain of individual carriers; (A) the 27 benchmark encodes, which measure frame-level distortion with ~22 luma flips per frame; and (C) an inverse check that sets the cap from a target PSNR, embeds and measures.

Experiment B: single flips. We encoded the first frame of *foreman* and *city* (CIF) with two x264 configurations—*ultrafast* (Intra16×16 only, deblocking off; the benchmark encoder) and *medium* (Intra4×4 and Intra16×16, deblocking on)—at QP 22, 28 and 34 (I-slice QP 19, 25, 31), sampled 100 candidates per encode (1,200 in total, skipping the rare carriers whose flip would add or remove an emulation-prevention

TABLE III

SINGLE-FLIP PROPAGATION GAIN G BY CARRIER CATEGORY
(EXPERIMENT B; n FLIPS, MEAN, MEDIAN, 90TH PERCENTILE).

Category	n	ultrafast			n	medium		
		mean	med.	p90		mean	med.	p90
Luma AC / 4×4	352	3.7	1.04	5.7	395	35.4	2.1	52.9
LumaDC	38	33.8	4.9	64.4	5	16.3	12.6	31.6
ChromaAC	173	9.2	1.12	11.9	146	4.4	1.06	5.4
ChromaDC	37	23.9	7.1	65.3	54	30.5	11.4	102.2
All luma	390	6.6	1.05	7.8	400	35.2	2.2	47.5

byte), flipped each one alone and decoded with and without the loop filter.

Direct energy. For all 385 sampled Intra4×4 luma carriers, the change of the flipped block’s own samples in the unfiltered decode equals, sample for sample, the change of the residual computed from Eqs. (6) and (3) with rounding; the decoder adds nothing else to the block. The ratio of that exact energy to the model value $4Q_{\text{step}}^2$ has median 1.012 and mean 1.022 (5th–95th percentile 0.95–1.19; small residuals are the most affected by rounding), and the pre-rounding factor κ of the sampled positions lies in $[1.000, 1.056]$, as Lemma 3 states.

Gains. Table III summarizes G . Three observations matter for the model. First, G does not depend on QP (luma mean 6.6, 7.4, 6.0 for ultrafast and 38.4, 33.3, 34.0 for medium at the three QPs), so distortion scales as Q_{step}^2 , i.e. by 6 dB per six QP steps. Second, the gain is heavy-tailed: the median carrier barely propagates ($G \approx 1$ –2), but single luma gains reach 309 (ultrafast) and 2,187 (medium), and the 10% largest gains account for 80% (ultrafast) and 84% (medium) of the total energy. Third, the encoder configuration matters more than anything else: the mean luma gain is 6.6 with Intra16×16 only and 35.2 with Intra4×4, whose nine directional modes chain many blocks. Deblocking changes the mean luma gain by about 1% (35.6 without, 35.2 with), so drift through intra prediction, not filtering, is the propagation mechanism.

Experiment A: sequences. For each of the 810 frames of the benchmark we located every changed bit in the bitstream, classified it with the parser, summed the model energy $4Q_{\text{step}}^2$ per plane and compared it with the measured SSE between the cover and stego decodes. Table IV reports the frame-level gain. Its *mean* (the ratio of total measured to total predicted energy) is the quantity Eq. (20) needs; for CIF it is 7.7, close to the single-flip mean of 6.6 measured on different frames in experiment B. More strikingly, Fig. 2 shows that drawing 22 single-flip gains at random reproduces the whole distribution of the CIF frame gains up to its 90th percentile (median 4.17 predicted vs. 4.00 measured); the measured upper tail is heavier (95th percentile 28.6 vs. 18.4), because carriers of the same frame share its content and are not independent. The gain grows with resolution (7.7, 17.3, 51.4). Our upscaled sequences contain large smooth regions, where Intra16×16 prediction carries a change across many MBs; native high-resolution content may behave differently, and calibrating $\bar{G}$ per resolution and encoder is necessary.

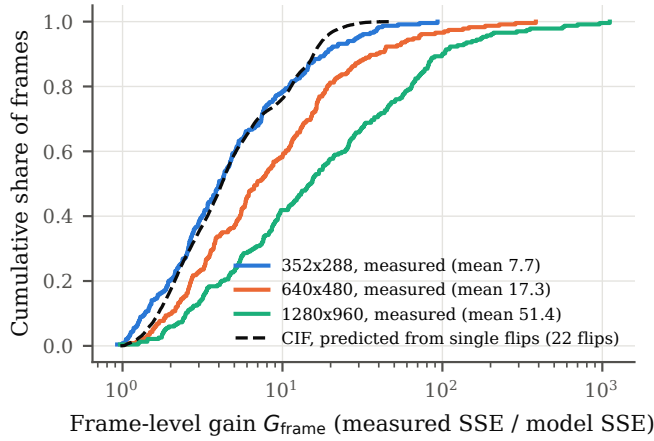


Fig. 2. Frame-level gain of the 702 benchmark frames with luma flips (experiment A), and its prediction for CIF from single flips alone (experiment B, 22 independent draws per frame).

TABLE IV
FRAME-LEVEL GAIN IN THE 27-ENCODE BENCHMARK (EXPERIMENT A, ULTRAFAST, 64 BITS PER IDR).

	CIF	640 × 480	1280 × 960
Frames with luma flips	234	234	234
Luma flips per frame	22.0	22.7	22.9
Mean gain $\bar{G}$	7.7	17.3	51.4
Frame gain, median	4.0	7.0	15.2
Frame gain, 95th pct.	28.6	65.8	173.8
Measured PSNR-Y, median [dB]	57.9	60.1	62.7
Per-frame error, MAE [dB]	4.1	5.1	6.7

Because a frame’s distortion is dominated by its few high-gain carriers, the model predicts the *distribution* of frame PSNR, not individual frames: with the per-resolution mean gain, the per-frame PSNR error has a mean absolute value of 4.1–6.7 dB and a correlation of only 0.19–0.35 with the measurement (leave-one-sequence-out errors are the same to 0.05 dB, so the gain does not overfit a sequence).

Experiment C: the inverse. We encoded 30 frames of *foreman* (CIF) with both configurations, computed the cap with Eq. (21) for $T = 40, 45$ and 50 dB under three choices of the gain, embedded a random payload filling the capacity, and repeated each setting with five independent keys and payloads (145 fully loaded frames per setting): (i) *expected*, the mean single-flip gain of experiment B, which makes the expected frame energy equal to the budget; (ii) *p95-MC*, the 95th percentile of the frame gain obtained by Monte Carlo from single flips; (iii) *p95-sequence* (ultrafast only), the 95th percentile of the measured CIF frame gains of experiment A. The single-flip gains are pooled over both sources and all three QPs, which Table III justifies for QP (the QP-19 subset has mean luma gain 6.6 and 38.4 against 6.6 and 35.2 pooled); the pool contains the first frame of *foreman* but none of the other 29 test frames. The third policy uses data that include these frames’ sequence. A cap at or above the number of candidates

TABLE V
INVERSE CHECK (EXPERIMENT C): GAIN USED, CAP, SHARE OF FRAMES WITH PSNR-Y $\geq T$ AND 5TH-PERCENTILE PSNR-Y [dB]; ^s: CAP AT OR ABOVE THE CANDIDATE COUNT OF AT LEAST 97% OF THE FRAMES’ IDRS (SATURATED).

Encoder	Policy	G	T	cap	$\geq T$	p05
medium	expected	35.2	45	200	61%	40.2
	expected	35.2	50	63	70%	45.1
	p95-MC	63.4	40	351	86%	38.3
	p95-MC	102.4	45	68	88%	43.2
	p95-MC	140.1	50	15	86%	43.6
ultrafast	expected	35.2	40	634 ^s	75%	37.3
	expected	6.6	50	311	46%	44.3
	p95-MC	15.8	50	130	80%	47.4
	p95-seq.	28.6	45	228	92%	43.7
	p95-seq.	28.6	50	72	91%	48.8
	all three	–	40	$\geq 721^s$	99%	41.5

in an IDR (about 650 for ultrafast and 460 for medium here) saturates: every candidate is used and the target no longer controls anything.

Table V and Fig. 4 give the outcome. The *expected* policy places the median frame near the target, as intended (46–70% of frames reach T). A percentile policy trades capacity for reliability: with the Monte Carlo percentile 80–88% of frames reach T , short of the nominal 95% for the reason seen in Fig. 2 (correlated carriers within a frame), while the percentile measured on sequences reaches 91–92%. Content is the other factor: the single-flip mean luma gain of *foreman* alone at QP 19 is about twice the pooled value (12.9 vs. 6.6 for ultrafast, 70 vs. 35 for medium, $n \approx 48$ each), so a gain calibrated on other content under-protects a sequence that propagates more. In every non-saturated setting the 5th percentile stays within 6.5 dB below the target, and the cap moves in the direction and by the amount the model predicts: from $T = 45$ to 50 dB the cap shrinks by $10^{0.5} \approx 3.2\times$ under the expected policy (200 to 63, 983 to 311). Saturation itself is informative: at the benchmark’s QP the ultrafast stream cannot drop below about 41.5 dB at its 5th percentile even when every candidate is used.

What the data suggest for carrier selection. Since a tenth of the carriers produces four fifths of the distortion, excluding high-gain carriers is worth more than reducing the payload. The single-flip data show which invariant features predict the gain. In the medium configuration, low-frequency trailing ones (coefficient positions with $i+j \leq 1$) have mean gains of 84–92 against 1.6 at $i+j = 6$, because low-frequency basis functions change the right column and bottom row from which later blocks predict. In both configurations, carriers in the top third of the picture have mean gains of 13.9 (ultrafast) and 65.6 (medium), against 2.2 and 6.1 in the bottom third, because more blocks are predicted from them. Restricting a random schedule to bottom-third carriers would reduce the expected energy by 4.8 dB (ultrafast) and 7.6 dB (medium) according to these averages, and excluding DC paths by 2.5 dB (ultrafast). These are projections from single-flip averages, not end-to-end measurements; the cost-aware schedule they motivate keeps

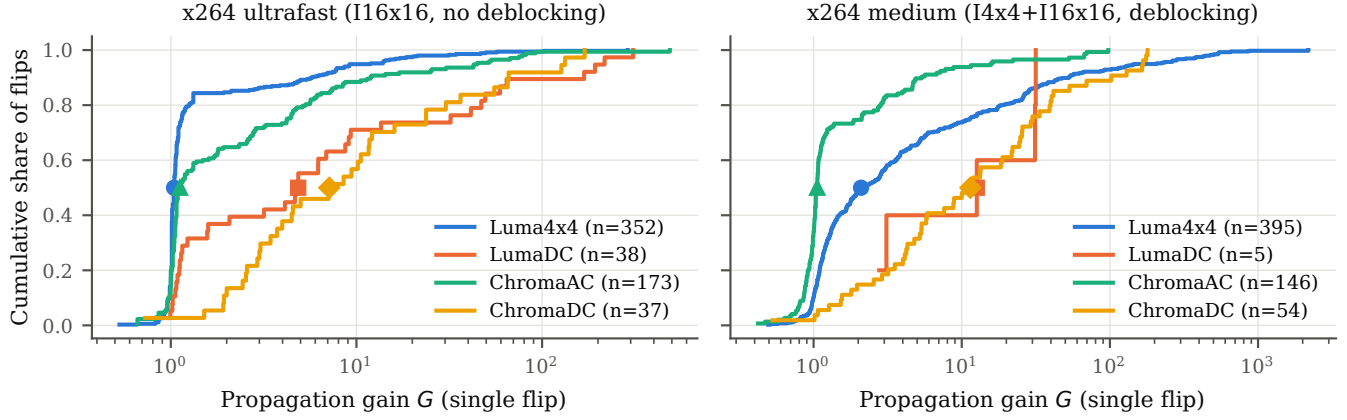


Fig. 3. Distribution of single-flip gains per carrier category for the two encoder configurations (markers: medians).

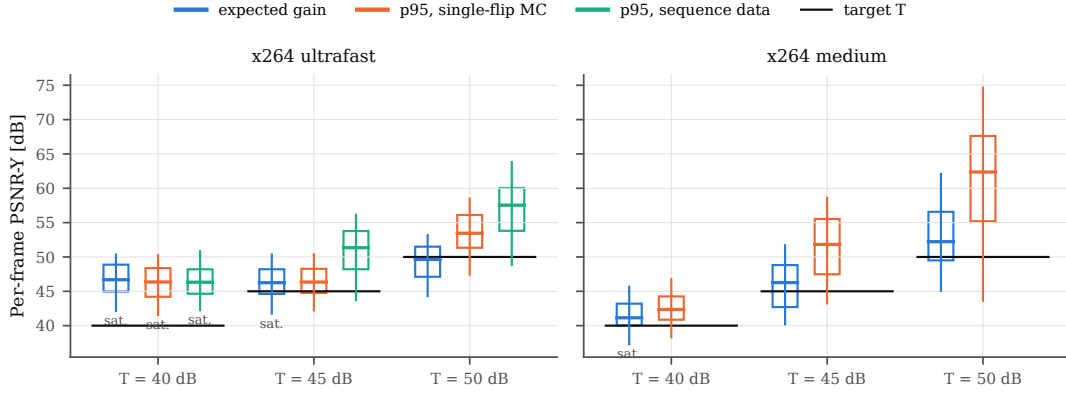


Fig. 4. Per-frame PSNR-Y for caps derived from a target T (boxes: quartiles; whiskers: 5th–95th percentiles; 145 frames per box).

the receiver-computability constraint of Section VII.

E. Sign statistics

Over all candidate signs of a case, the share of negative trailing ones is 0.504 before and after embedding; the two-proportion z -statistic is at most 0.223 in absolute value over the 27 cases, far from the 1.96 threshold of a 5% test. This is a first-order indicator only (Section X).

F. Performance

Table VI compares Groth16 and PLONK on the same R1CS and witness; both verify a correct proof and reject a tampered public input. Groth16’s 129-byte proof is what makes the covert channel practical: snarkjs serializes a PLONK proof as 2.25 KB of JSON, which would need about 290 IDRs at the default cap, and even a compact binary encoding is several times larger than 129 B. Table VII gives the end-to-end stages for 300 CIF frames. Native costs scale with the number of parsed IDRs: the median capacity scan takes 52, 220 and 770 ms per IDR at the three resolutions, embedding 61, 245 and 891 ms per carrying IDR, and the digest, which parses the 26 carrying IDRs, 1.6, 6.4 and 22.6 s. Parsing, not

TABLE VI
PROOF SYSTEMS ON THE CAMERA_VIDEO CIRCUIT (8,737 CONSTRAINTS, 3 PUBLIC INPUTS).

	Groth16	PLONK
Trials	5	3
Prove [s]	1.52–1.60	21.5–23.3
Verify [s]	0.91–1.01	0.90–0.94
Proof size	129 B (compressed)	2.25 KB (JSON)
Peak prover RSS [MB]	695	864
Setup	circuit-specific (Phase 2)	universal SRS, 3.2 s
Proving key	4.2 MB	30.1 MB

proving, bounds throughput at high resolution; the parser is single-threaded and unoptimized.

G. Attack matrix

Table VIII lists the attacks run through the service’s verify job on the end-to-end stream. Only the untouched stream is accepted. Notably, flipping a sign that does not carry the frame—in an IDR that does carry it—is rejected through the digest, and so is the same valid payload embedded into another video; a camera that builds its own registry produces a proof

TABLE VII
END-TO-END STAGES, 300 CIF FRAMES, 5 TRIALS (MEDIAN, MS).

Sender		Receiver	
Cover digest	511	Extract	535
Groth16 prove	1,774	Digest + verify	1,448
Embed	500	Strict decode	150
Total	2,779	Total	1,989

TABLE VIII
ATTACK MATRIX (VERIFY JOB OUTCOME AND REASON).

Case	Result (rejection reason)
Untouched stego stream	accepted
Wrong stego key	payload_not_found
Flip a scheduled carrier sign	malformed_proof_payload
Flip non-carrier sign, carrier IDR	proof_invalid
Flip a sign in a later frame	proof_invalid
Drop the first IDR	payload_not_found
Truncate to half	proof_invalid
Re-encode (transcode)	payload_not_found
Swap the message	proof_invalid
Camera outside the registry	proof_invalid
Replay into another video	proof_invalid

that verifies against its own root but not against the trusted one.

X. DISCUSSION AND LIMITATIONS

Where the design fits. The scheme protects bitstreams that are moved without re-encoding: surveillance and body-camera evidence chains that archive the camera’s H.264, files exchanged as files, and newsroom or legal workflows that need to show that footage comes from an authorized device without naming it. In these settings metadata is routinely lost while the elementary stream survives, and hiding the proof keeps an adversary from selectively removing it.

Why not a SEI message? A proof could be carried in a user-data SEI NAL unit at no capacity cost and with the same digest construction (the SEI unit would be excluded instead of the carrier bits). The difference is covertness: a SEI payload is visible to every parser and trivially stripped by a sanitizer, while the sign channel is located only with K . When covertness is not needed, the SEI variant is simpler, and our binding and proof carry over unchanged.

Verification requires the stego key. Both extraction and the carrier mask depend on K , so verification is limited to key holders; a key holder still cannot forge proofs (G3). Public verification would require publishing the carrier set, e.g. as a commitment opened to verifiers, which trades covertness for verifiability.

Fragility. Any re-encode, resize or transcode destroys the payload and changes the digest. Robust provenance across edits requires transformation proofs [4], [6], [8] or a robust watermark acting as a soft binding [1]; combining such a layer with an in-band anonymous proof is an open direction.

What the proof does not show. A registered camera can film a screen; the camera secret lives in software in our

prototype, so a compromised device can sign anything; registry governance (enrollment, revocation by publishing a new root, root distribution) is outside the system. Binding the secret to a secure element or an attested sensor pipeline, as in ZK-IMG [5], would address part of this.

Live streams. In mode 1 the proof binds the message only and can be replayed; video binding for live streams would need a proof that commits to frames embedded later, e.g. a chain of proofs or a final proof carried in a trailing segment.

Codec scope. The channel needs CAVLC and IDR frames. Most capture devices use the High profile with CABAC, where sign bits are bypass-coded but not length-invariant in the same simple way; HEVC uses sign data hiding, which interacts with sign embedding. Porting the channel is engineering work with its own distortion and syntax analysis.

Trusted setup. Groth16 needs a circuit-specific ceremony; our ceremony is a demonstration. PLONK removes the circuit-specific phase at a cost of roughly $14\times$ in proving time and a 2.25 KB proof (Section IX), which no longer fits the covert channel at the default cap.

Steganalysis. We only measured first-order sign statistics. The flips are sparse (about 32 per frame in our runs) but the drift they cause is visible in the pixel domain as low-amplitude block-shaped changes; rich-model and learned steganalysis [41], [42] should be evaluated before claiming covertness against an active warden. The cost-aware schedule of Section VII is also a natural defense.

XI. CONCLUSION

We showed that a zero-knowledge proof of anonymous camera membership is small enough to be hidden in the trailing-one signs of an H.264 stream, and that it can be bound to the exact stream that carries it through a digest that masks only the keyed carrier positions. The resulting system needs no metadata, hides the signer within a registry, rejects replay, truncation, edits and re-encodes, and lets a verifier who holds the stego key check a proof that it cannot forge. We also derived a closed-form distortion model of sign embedding—one flip costs $4Q_{\text{step}}^2$ up to a factor in $[0.925, 1.057]$ —and turned it into a per-frame bit budget for a target PSNR, which our experiments confirm once the propagation gain of the encoder configuration is known. Future work includes minimal-distortion coding with the derived costs, steganalysis, CABAC and HEVC, hardware-bound camera secrets and video binding for live streams.

ARTIFACT AVAILABILITY

Source code, circuit, benchmark scripts and the recorded results used in this paper are available at <https://github.com/Saudadeeee/zk-snark-X-Steganography>.

REFERENCES

- [1] Coalition for Content Provenance and Authenticity, “Content credentials: C2PA technical specification, version 2.4,” Apr. 2026. [Online]. Available: https://spec.c2pa.org/specifications/specifications/2.4/specs/C2PA_Specification.html

- [2] E. Ben-Sasson, A. Chiesa, C. Garman, M. Green, I. Miers, E. Tromer, and M. Virza, “Zerocash: Decentralized anonymous payments from Bitcoin,” in *Proceedings of the 2014 IEEE Symposium on Security and Privacy (SP)*, 2014, pp. 459–474.
- [3] Privacy & Scaling Explorations, “Semaphore: A zero-knowledge protocol for anonymous interactions,” Project website and GitHub repository, source: <https://github.com/semaphore-protocol/semaphore>, accessed Oct. 2026. [Online]. Available: <https://semaphore.pse.dev>
- [4] A. Naveh and E. Tromer, “PhotoProof: Cryptographic image authentication for any set of permissible transformations,” in *Proceedings of the 2016 IEEE Symposium on Security and Privacy (SP)*, 2016, pp. 255–271.
- [5] D. Kang, T. Hashimoto, I. Stoica, and Y. Sun, “ZK-IMG: Attested images via zero-knowledge proofs to fight disinformation,” arXiv:2211.04775, 2022. [Online]. Available: <https://arxiv.org/abs/2211.04775>
- [6] T. Datta, B. Chen, and D. Boneh, “VerITAS: Verifying image transformations at scale,” in *Proceedings of the 2025 IEEE Symposium on Security and Privacy (SP)*, 2025, pp. 4606–4623.
- [7] S. Dziembowski, S. Ebrahimi, and P. Hassanizadeh, “VIMz: Private proofs of image manipulation using folding-based zkSNARKs,” *Proceedings on Privacy Enhancing Technologies*, vol. 2025, no. 2, pp. 344–362, 2025.
- [8] A. Frolov, J. Guo, X. Zhao, T. Datta, D. Boneh, and I. Miers, “zk-Cinema: Proving video provenance in zero knowledge,” Cryptology ePrint Archive, Paper 2026/1598, 2026. [Online]. Available: <https://eprint.iacr.org/2026/1598>
- [9] L. Yu and Y. Yao, “AIPA: Anonymous image provenance authentication in online social networks via unlinkable pseudonym certificates and zk-SNARKs,” Cryptology ePrint Archive, Paper 2026/1914, 2026. [Online]. Available: <https://eprint.iacr.org/2026/1914>
- [10] G. J. Simmons, “The prisoners’ problem and the subliminal channel,” in *Advances in Cryptology: Proceedings of Crypto 83*. Plenum Press, 1984, pp. 51–67.
- [11] C. Cachin, “An information-theoretic model for steganography,” in *Information Hiding (IH 1998)*, ser. Lecture Notes in Computer Science, vol. 1525. Springer, 1998, pp. 306–318.
- [12] J. Fridrich, *Steganography in Digital Media: Principles, Algorithms, and Applications*. Cambridge University Press, 2009.
- [13] S. K. Kapotas, E. E. Varsaki, and A. N. Skodras, “Data hiding in H.264 encoded video sequences,” in *Proceedings of the 9th IEEE Workshop on Multimedia Signal Processing (MMSP)*, Oct. 2007, pp. 373–376.
- [14] B. G. Mobasser and Y. Naik Raikar, “Authentication of H.264 streams by direct watermarking of CAVLC blocks,” in *Security, Steganography, and Watermarking of Multimedia Contents IX*, ser. Proc. SPIE, vol. 6505. SPIE, Feb. 2007, p. 65051W.
- [15] S.-J. Horng, M. E. Farfoura, P. Fan, X. Wang, T. Li, and J.-M. Guo, “A low cost fragile watermarking scheme in H.264/AVC compressed domain,” *Multimedia Tools and Applications*, vol. 72, no. 3, pp. 2469–2495, 2014.
- [16] X. Ma, Z. Li, H. Tu, and B. Zhang, “A data hiding algorithm for H.264/AVC video streams without intra-frame distortion drift,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 20, no. 10, pp. 1320–1330, Oct. 2010.
- [17] M. M. Yeung and F. Mintzer, “An invisible watermarking technique for image verification,” in *Proceedings of the International Conference on Image Processing (ICIP)*, vol. 2, 1997, pp. 680–683.
- [18] P. W. Wong, “A public key watermark for image verification and authentication,” in *Proceedings of the 1998 International Conference on Image Processing (ICIP)*, vol. 1, 1998, pp. 455–459.
- [19] A. Adelsbach and A.-R. Sadeghi, “Zero-knowledge watermark detection and proof of ownership,” in *Information Hiding (IH 2001)*, ser. Lecture Notes in Computer Science, vol. 2137. Springer, 2001, pp. 273–288.
- [20] M. Toso and S. B. Chung, “Combining steganography and zero-knowledge proofs to embed and prove a digital signature in an image,” in *Proceedings of the Midwest Instruction and Computing Symposium (MICS)*, 2006. [Online]. Available: https://www.micsymposium.org/mics_2006/papers/TosoAndChung.pdf
- [21] R. C. Merkle, “A digital signature based on a conventional encryption function,” in *Advances in Cryptology – CRYPTO ’87*, ser. Lecture Notes in Computer Science, vol. 293. Springer, 1988, pp. 369–378.
- [22] L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, and M. Schofnegger, “Poseidon: A new hash function for zero-knowledge proof systems,” in *Proceedings of the 30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, 2021, pp. 519–535. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity21/presentation/grassi>
- [23] A. Westfield, “F5—a steganographic algorithm: High capacity despite better steganalysis,” in *Information Hiding (IH 2001)*, ser. Lecture Notes in Computer Science, vol. 2137. Springer, 2001, pp. 289–302.
- [24] T. Filler, J. Judas, and J. Fridrich, “Minimizing additive distortion in steganography using syndrome-trellis codes,” *IEEE Transactions on Information Forensics and Security*, vol. 6, no. 3, pp. 920–935, Sep. 2011.
- [25] J. Fridrich and T. Filler, “Practical methods for minimizing embedding impact in steganography,” in *Security, Steganography, and Watermarking of Multimedia Contents IX*, ser. Proc. SPIE, vol. 6505. SPIE, Feb. 2007, p. 650502.
- [26] ITU-T, “H.264: Advanced video coding for generic audiovisual services,” International Telecommunication Union, Recommendation ITU-T H.264 (V16), Jun. 2026. [Online]. Available: <https://www.itu.int/rec/T-REC-H.264>
- [27] T. Wiegand, G. J. Sullivan, G. Bjøntegaard, and A. Luthra, “Overview of the H.264/AVC video coding standard,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 13, no. 7, pp. 560–576, Jul. 2003.
- [28] I. E. Richardson, *The H.264 Advanced Video Compression Standard*, 2nd ed. John Wiley & Sons, 2010.
- [29] H. S. Malvar, A. Hallapuro, M. Karczewicz, and L. Kerofsky, “Low-complexity transform and quantization in H.264/AVC,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 13, no. 7, pp. 598–603, Jul. 2003.
- [30] P. S. L. M. Barreto and M. Naehrig, “Pairing-friendly elliptic curves of prime order,” in *Selected Areas in Cryptography (SAC 2005)*, ser. Lecture Notes in Computer Science, vol. 3897. Springer, 2006, pp. 319–331.
- [31] J. Groth, “On the size of pairing-based non-interactive arguments,” in *Advances in Cryptology – EUROCRYPT 2016, Part II*, ser. Lecture Notes in Computer Science, vol. 9666. Springer, 2016, pp. 305–326.
- [32] S. Bowe, A. Gabizon, and I. Miers, “Scalable multi-party computation for zk-SNARK parameters in the random beacon model,” Cryptology ePrint Archive, Paper 2017/1050, 2017. [Online]. Available: <https://eprint.iacr.org/2017/1050>
- [33] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, “PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge,” Cryptology ePrint Archive, Paper 2019/953, 2019. [Online]. Available: <https://eprint.iacr.org/2019/953>
- [34] M. Bellés-Muñoz, M. Isabel, J. L. Muñoz-Tapia, A. Rubio, and J. Baylina, “Circom: A circuit description language for building zero-knowledge applications,” *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 6, pp. 4733–4751, 2023.
- [35] iden3, “snarkjs: JavaScript and WebAssembly implementation of zk-SNARK and PLONK schemes,” GitHub repository, accessed Oct. 2026. [Online]. Available: <https://github.com/iden3/snarkjs>
- [36] H. Krawczyk and P. Eronen, “HMAC-based extract-and-expand key derivation function (HKDF),” RFC 5869, May 2010. [Online]. Available: <https://www.rfc-editor.org/info/rfc5869>
- [37] K. Bagheri, M. Kohlweiss, J. Siim, and M. Volkhov, “Another look at extraction and randomization of Groth’s zk-SNARK,” in *Financial Cryptography and Data Security (FC 2021)*, ser. Lecture Notes in Computer Science, vol. 12674. Springer, 2021, pp. 457–475.
- [38] iden3, “Prepared phase-2 powers of tau files for BN128 (powersOfTau28_hez_final_*.ptau), 54 contributions and a random beacon,” Listed in the snarkjs README, files hosted at <https://circom.info/>, accessed Oct. 2026. [Online]. Available: <https://github.com/iden3/snarkjs>
- [39] Xiph.Org, “Xiph.org video test media [derf’s collection],” accessed Oct. 2026. [Online]. Available: <https://media.xiph.org/video/derf/>
- [40] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: From error visibility to structural similarity,” *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, Apr. 2004.
- [41] J. Fridrich and J. Kodovský, “Rich models for steganalysis of digital images,” *IEEE Transactions on Information Forensics and Security*, vol. 7, no. 3, pp. 868–882, Jun. 2012.
- [42] A. D. Ker, P. Bas, R. Böhme, R. Cogan, S. Craver, T. Filler, J. Fridrich, and T. Pevný, “Moving steganography and steganalysis from the laboratory into the real world,” in *Proceedings of the 1st ACM Work-*

APPENDIX A WORKED EXAMPLE OF ONE CARRIER

We follow one carrier of a demonstration run (akiyo, CIF, x264 default preset, I-slice QP 19, so $Q_{\text{step}} = 5.5$ and $4Q_{\text{step}}^2 = 121$) from the bitstream to the pixels. The carrier is the first trailing-one sign of luma block 5 of MB 212 (Intra4×4, prediction mode Diagonal_Down_Right), at RBSP bit 57,561 of the first IDR NAL unit.

CAVLC. With $n_C = 6$ the block is coded as coeff_token=1011 (TotalCoeff 4, TrailingOnes 3), sign flags 011, one level 1, total_zeros=111 and two run_before codes. The scheduled bit is the first sign flag; whitening requires a 1, so the flag becomes 1 and the flags read 111. All other bits keep their position and meaning. In the file the byte at offset 7,805 changes from bf to ff.

Levels and scaling. The scan (1, −1, 0, −1, 1, 0, ...) becomes (1, −1, 0, −1, −1, 0, ...): zig-zag position 4, raster (1, 1), an odd/odd coefficient. With $v_1(1, 1) = 18$ and $2^{\lfloor 19/6 \rfloor} = 8$ the scaled coefficient changes from 144 to −144:

$$D = \begin{bmatrix} 88 & -112 & 0 & 0 \\ 0 & \pm 144 & 0 & 0 \\ -88 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

Residual. The inverse transform gives

$$\hat{e}_{\text{before}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 2 & 2 & 3 & 3 \\ 0 & 1 & 4 & 6 \\ -4 & -2 & 2 & 4 \end{bmatrix}, \quad \hat{e}_{\text{after}} = \begin{bmatrix} -4 & -2 & 2 & 4 \\ 0 & 1 & 4 & 6 \\ 2 & 2 & 3 & 3 \\ 1 & 0 & 0 & 0 \end{bmatrix},$$

a change of energy $\sum \Delta \hat{e}^2 = 128$. Lemma 3 predicts $4Q_{\text{step}}^2 \kappa_1(1, 1) = 121 \times 1.046 = 126.6$ before rounding.

Reconstruction. A flip does not affect blocks decoded earlier, so if this were the only carrier the block’s prediction would be unchanged and its samples would change by exactly $\Delta \hat{e}$ (Experiment B confirms this for every sampled Intra4×4 block). In the demo a block decoded earlier also carried a flip, which shows up as a uniform change of +1 in this block’s prediction: an instance of intra drift. Our Python reconstruction (prediction plus residual) matches FFmpeg sample by sample before and after embedding, decoded with `-skip_loop_filter all`; deblocking then smooths the block edges.

APPENDIX B NOTATION

K, s	stego key, camera secret
$k_{\text{sch}}, k_{\text{wh}}$	schedule and whitening subkeys
ρ, pk	registry root, camera public key Poseidon(s)
$\Gamma_K(V)$	carrier positions of stream V under K
$\text{dig}_K, \text{bind}$	carrier-masked digest, binding
F, P, π	frame, payload, compressed proof
cap	bits per IDR segment
$Q_{\text{step}}, v_m(i, j)$	quantizer step, scaling factor
κ, G	position factor, propagation gain